Colab 連結:
https://colab.research.google.com/drive/1AvKUfqU-4r3NtyNeic_gxuS3W4U2ur45?usp=sharing

一、課程內容

1、匯入重要的pthon套件.

- numpy:算數學時的套件
- pandas:Python的Excel
- matplotllib:畫圖元件

2. 畫一個函數圖形

f(x) = ax+b

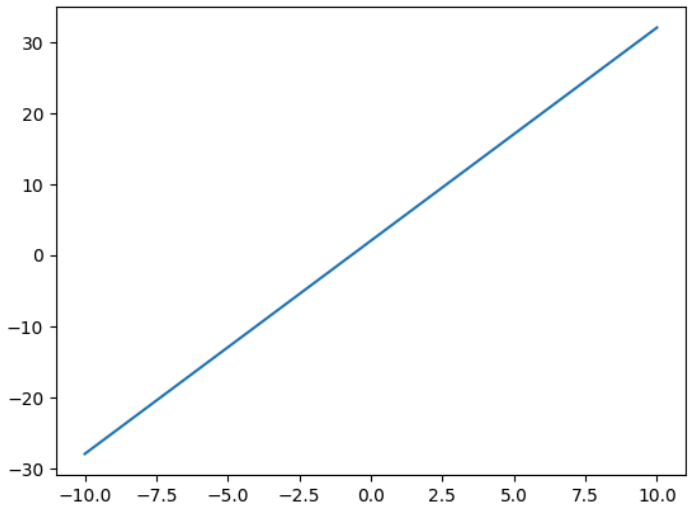
$f(x) = 3x + 2$ 的圖形

```
%matplotlib inline

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

x = np.linspace(-10, 10, 100)
y = 3*x+2
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D at 0x7c504d390690>]
```



二、作業

本練習題將呈現運用python表現簡單迴歸與多元迴歸圖以及迴歸殘差圖。

迴歸分析的核心目的，是藉由一個或多個自變數（X）的變化，來解釋並預測應變數（Y）的平均表現。

> ### Part I 簡單迴歸（Simple Linear Regression）

當只有一個自變數 X，且關係為直線時:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Y：應變數（預測目標）

X：自變數（輸入特徵）

β_0：截距（Intercept）

β_1：斜率 / 迴歸係數（Regression Coefficient）

ε：隨機誤差項（Error Term），通常假設其符合獨立同分布的正態分布

> ### Part II 多元線性迴歸（Multiple Linear Regression）

當有兩個或以上的自變數時:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_k X_k + \varepsilon$$

用矩陣形式（Matrix Form）可以簡寫為：

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

Part III 邏輯斯迴歸（Logistic Regression）用於分類問題（預測機率）

其迴歸函數是將線性組合套入 Sigmoid 函數中：

$$P(Y = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \cdots + \beta_k X_k)}}$$

```
# Part I 簡單迴歸
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. 準備數據（X：讀書時數，y：考試分數）
X = np.array([1, 2, 3, 4, 5]).reshape(-1, 1)    # 轉為二維陣列
y = np.array([50, 55, 65, 70, 85])

# 2. 訓練迴歸模型
model = LinearRegression()
model.fit(X, y)

# 3. 產生用來畫「迴歸直線」的預測點
X_line = np.linspace(1, 5, 100).reshape(-1, 1)    # 在 1 到 5 之間均勻產生 100 個點
y_line = model.predict(X_line)                    # 計算對應的預測 y 值

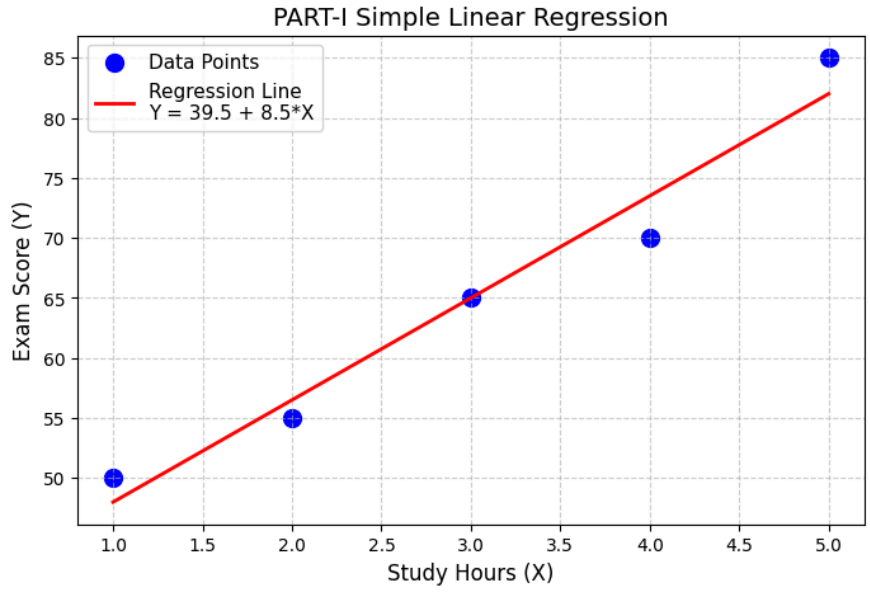
# 4. 開始繪圖
plt.figure(figsize=(8, 5))

# 畫出原始數據點（藍色散佈圖）
plt.scatter(X, y, color='blue', label='Data Points', s=100)

# 畫出迴歸直線（紅色實線）
plt.plot(X_line, y_line, color='red', linewidth=2, label=f'Regression Line\nY = {model.intercept_:.1f} + {model.coef_[0]:.1f}*X')

# 5. 圖表美化（設定標籤與標題）
plt.title('PART-I Simple Linear Regression', fontsize=14)
plt.xlabel('Study Hours (X)', fontsize=12)
plt.ylabel('Exam Score (Y)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.6)    # 加上網格線
plt.legend(fontsize=11)                      # 顯示圖例

# 6. 顯示圖表
plt.show()
```



PART-I 簡單線性迴歸圖形說明

本圖呈現 「單一輸入特徵（讀書時數）」與「連續輸出結果（考試分數）」之間的線性趨勢與預測關係。

---

1. 藍色散佈點（Data Points）：真實觀測值

- **代表數據**：收集到的 5 筆樣本 (1小時，50分)、(2小時，55分)、(3小時，65分)、(4小時，70分)、(5小時，85分)。

- **分佈特徵**：點的分佈呈現由左下往右上傾斜，顯示「讀書時數」與「考試分數」存在明顯的**強正相關。**

---

2. 紅色實線（Regression Line）：最佳擬合線

模型透過最小平方法（OLS）計算出的具體方程式為：

$$Y = 40.0 + 8.5X$$

- **截距** $\beta_0 = 40.0$：當 $X = 0$（完全沒讀書）時的基準預測分數為 **40 分。**
- **斜率** $\beta_1 = 8.5$：每多讀書 **1 小時**，考試分數平均提升 **8.5 分。**
- **幾何意義**：此直線是所有可能線條中，讓「所有點到直線之垂直距離平方和（SSE）」達到最小的最佳估計線。

---

3. 點與線的垂直距離：誤差項（Residuals）

藍點與紅線之間的垂直差距即為**殘差**，反映無法被「讀書時數」完全解釋的隨機因素（如考試運氣、專注度）：

讀書時數 (X)	實際分數 (Y)	模型預測值 ($\hat{Y}$)	殘差 ($Y - \hat{Y}$)
1	50	48.5	+1.5
2	55	57.0	-2.0
3	65	65.5	-0.5
4	70	74.0	-4.0
5	85	82.5	+2.5

---

4. 實務應用價值

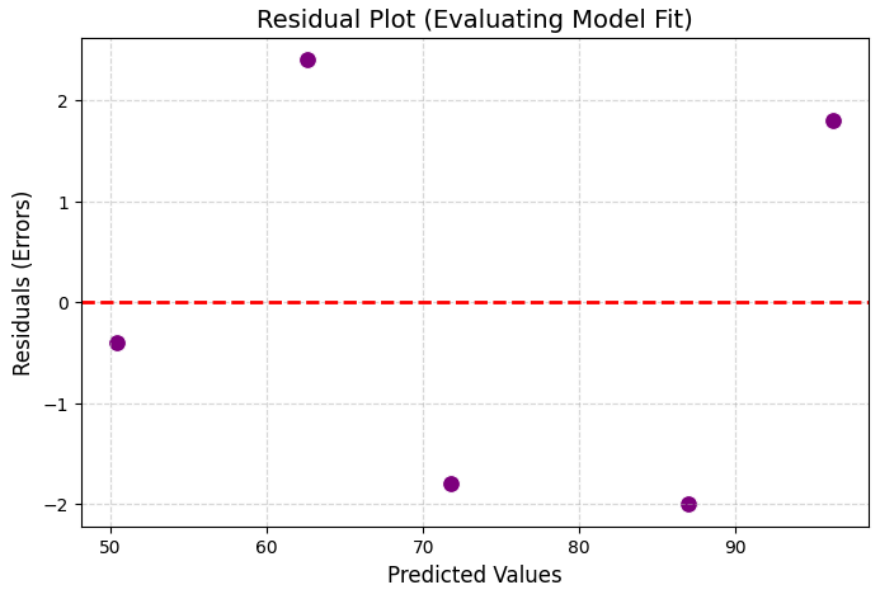
- **定量內插預測**：若投入 $X = 3.5$ 小時，代入公式即可推算期望分數為：
$$Y = 40.0 + 8.5 \times 3.5 = 69.75 \text{ 分}$$
- **量化邊際效應**：直接以斜率評估付出成本（時間）與產出（分數）的轉換率（1 小時換 8.5 分）。

```
import seaborn as sns

# 計算預測值與殘差
y_pred = model.predict(X)
residuals = y - y_pred

# 繪製殘差圖
plt.figure(figsize=(8, 5))
sns.scatterplot(x=y_pred, y=residuals, color='purple', s=100)
plt.axhline(y=0, color='red', linestyle='--', linewidth=2)  # 基準線

plt.title('Residual Plot (Evaluating Model Fit)', fontsize=14)
plt.xlabel('Predicted Values', fontsize=12)
plt.ylabel('Residuals (Errors)', fontsize=12)
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()
```



殘差圖（Residual Plot）圖形說明

本圖用來於檢視模型是否滿足線性迴歸的基本假設，並確認「線性模型」是否合適。

---

1. 座標軸與元素定義

- **水平軸（Predicted Values, $\hat{Y}$）**：模型對每個樣本計算出的預測分數。
- **垂直軸（Residuals, $e = Y - \hat{Y}$）**：真實值與預測值的差距（誤差項）。
- **紅色虛線（$y = 0$ 基準線）**：代表「預測完全準確、零誤差」的理想位置。
- **紫色散佈點**：每個樣本的殘差落點：
  - 點落在紅線**上方**（$e > 0$）：代表模型**低估**了真實分數（真實值大於預測值）。
  - 點落在紅線**下方**（$e < 0$）：代表模型**高估**了真實分數（真實值小於預測值）。

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. 直接建立數據（X1: 讀書時數, X2: 出席率, y: 分數）
X1 = np.array([1, 2, 3, 4, 5])
X2 = np.array([60, 70, 75, 90, 95])
y = np.array([50, 65, 70, 85, 98])

# 將兩個特徵組合成一個二維陣列的特徵矩陣
X = np.column_stack((X1, X2))

# 2. 訓練多元線性迴歸模型
model = LinearRegression()
model.fit(X, y)

# 3. 建立 3D 網格空間，用來繪製「迴歸平面」
x1_range = np.linspace(1, 5, 20)
x2_range = np.linspace(50, 100, 20)
X1_mesh, X2_mesh = np.meshgrid(x1_range, x2_range)

# 將網格點拉平，丟入模型預測以得到平面上的 Z 軸高度 (y_pred)
X_mesh_flat = np.column_stack((X1_mesh.ravel(), X2_mesh.ravel()))
y_pred_mesh = model.predict(X_mesh_flat).reshape(X1_mesh.shape)

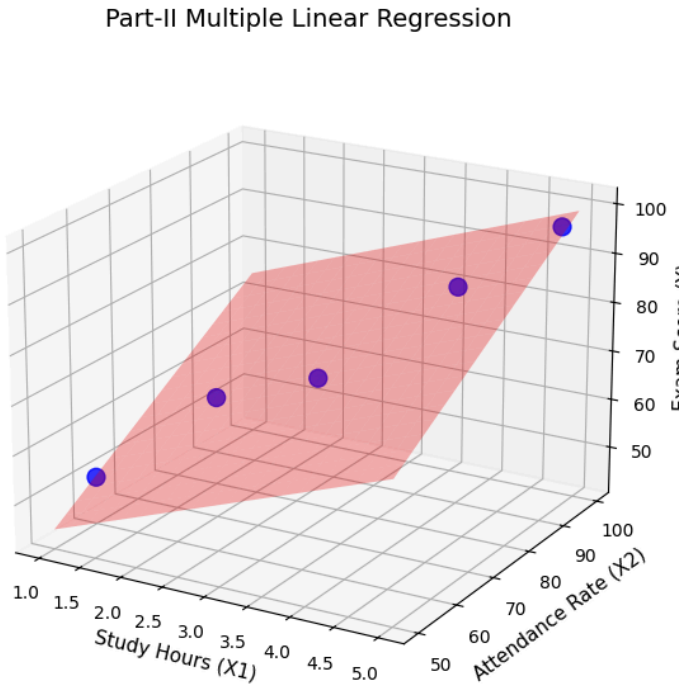
# 4. 開始畫 3D 圖
fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')

# 畫出真實的數據點 (藍色散佈點)
ax.scatter(X1, X2, y, color='blue', s=100, label='Actual Data', alpha=0.8)

# 畫出預測的迴歸平面 (紅色半透明平面)
surf = ax.plot_surface(X1_mesh, X2_mesh, y_pred_mesh, color='red', alpha=0.3, edgecolor='none')

# 5. 設定標籤與視覺優化
ax.set_title('Part-II Multiple Linear Regression', fontsize=14)
ax.set_xlabel('Study Hours (X1)', fontsize=11)
ax.set_ylabel('Attendance Rate (X2)', fontsize=11)
ax.set_zlabel('Exam Score (Y)', fontsize=11)
ax.view_init(elev=20, azim=-60)  # 調整視角

plt.show()
```



Part-II 多元線性迴歸 3D 圖形意義說明

3D 圖形展示了兩個自變數（讀書時數 X_1、出席率 X_2）如何共同決定連續應變數（考試分數 Y）的空間關係。

---

1. 藍色散佈點（Actual Data）：真實觀測值

- **空間座標**：每個點代表一位學生的三維數據 (X_1, X_2, Y)，例如 (1小時，60%出席，50分) 至 (5小時，95%出席，98分)。
- **分佈特徵**：點群在 3D 空間中由左下方往右上方延伸，顯示考試分數隨讀書時數增加、出席率提升而同步上升。

---

2. 紅色半透明平面（Regression Plane）：最佳擬合平面

在單變數迴歸中擬合的是**一條線**；在兩個自變數的多元迴歸中，擬合對象升維為**一個平面**：

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2$$

經最小平方法（OLS）計算出的模型參數為：

- **截距** $\beta_0 \approx -8.45$：兩項特徵皆為 0 時的理論基準值。
- **斜率** $\beta_1 \approx 4.02$ **（讀書時數係數）**：在**控制出席率不變**的前提下，每多讀書 1 小時，預測分數平均提升約 **4.02 分**。
- **斜率** $\beta_2 \approx 0.91$ **（出席率係數）**：在**控制讀書時數不變**的前提下，出席率每提升 1%，預測分數平均提升約 **0.91 分**。
- **平面的幾何意義**：所有可能的平面中，使 5 個真實資料點到平面的「垂直高度差平方和（SSE）」最小的唯一解。

---

3. 點與平面的垂直間隙：殘差（Residuals）

- 資料點懸浮在平面上方或陷在平面下方，其與平面的垂直距離即為**誤差項** ε。
- 範例中資料點極度貼近該紅色平面，代表這兩個特徵能解釋絕大部分的分數變異（模型解釋力 R^2 極高）。

---

4. 實務與分析用途

- **控制變因（Ceteris Paribus）效果評估**：平面的雙向傾斜角度分別量化了「時數」與「出席率」對分數的獨立邊際貢獻。
- **雙變數聯合預測**：給定任意組合（例如讀書 3.5 小時、出席率 80%），對應至平面上的 Z 軸高度即可獲得預測分數：

$$Y = -8.45 + 4.02 \times 3.5 + 0.91 \times 80 \approx 78.42 \text{ 分}$$

```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression

# 1. 產生 2 個特徵的二元分類資料，方便在平面繪圖
X, y = make_classification(
        n_samples=200,
        n_features=2,
        n_informative=2,
        n_redundant=0,
        n_clusters_per_class=1,
        random_state=42,
)

model = LogisticRegression()
model.fit(X, y)

# 2. 設定畫布
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# --- 圖 1：Sigmoid 曲線與投影機率 ---
# 計算每個樣本的線性組合 z = β0 + β1*x1 + β2*x2
z = model.decision_function(X)
prob = model.predict_proba(X)[:, 1]

# 繪製理論上的 Sigmoid 曲線
z_range = np.linspace(-6, 6, 300)
sigmoid = 1 / (1 + np.exp(-z_range))

axes[0].plot(z_range, sigmoid, color="black", label="Sigmoid Curve")
axes[0].scatter(z, y, c=y, cmap="coolwarm_r", alpha=0.6, edgecolors="k", label="Actual Label (0 or 1)")
axes[0].scatter(z, prob, c=y, cmap="coolwarm_r", marker="x", alpha=0.5, label="Predicted Prob")
axes[0].axhline(0.5, color="gray", linestyle="--", linewidth=1)
axes[0].set_title("Sigmoid Function & Prediction")
axes[0].set_xlabel(r"$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2$")
axes[0].set_ylabel("Probability")
axes[0].legend()
axes[0].grid(True, alpha=0.3)

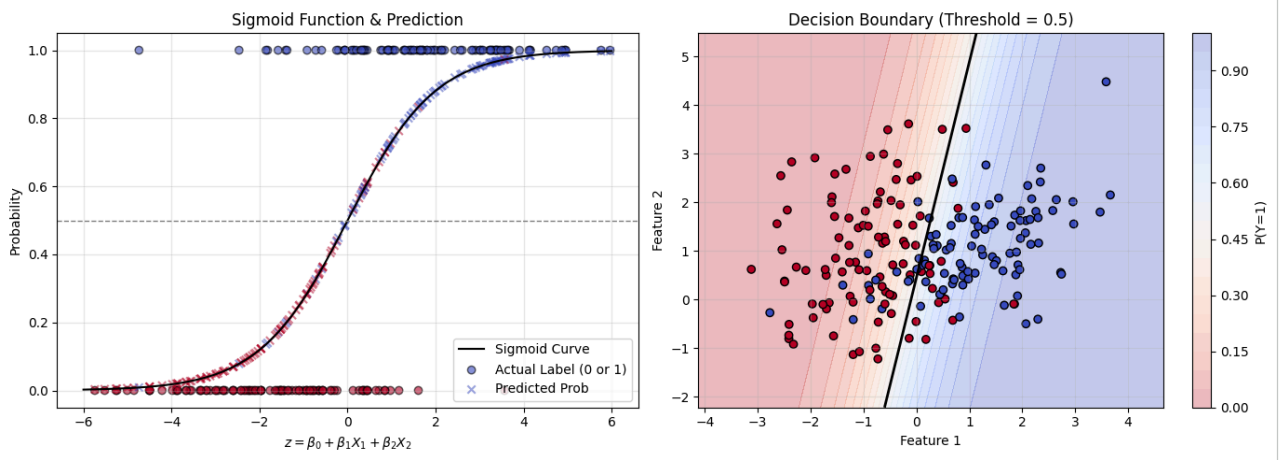
# --- 圖 2：二維空間決策邊界 ---
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 200), np.linspace(y_min, y_max, 200))

# 預測網格上每個點的機率
Z = model.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]
Z = Z.reshape(xx.shape)
```

```
# 繪製機率等高線與決策邊界（機率 = 0.5）
contour = axes[1].contourf(xx, yy, Z, levels=20, cmap="coolwarm_r", alpha=0.3)
axes[1].contour(xx, yy, Z, levels=[0.5], colors="black", linewidths=2)
fig.colorbar(contour, ax=axes[1], label="P(Y=1)")

# 繪製原始資料點
scatter = axes[1].scatter(X[:, 0], X[:, 1], c=y, cmap="coolwarm_r", edgecolors="k")
axes[1].set_title("Decision Boundary (Threshold = 0.5)")
axes[1].set_xlabel("Feature 1")
axes[1].set_ylabel("Feature 2")
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



PART-III 兩張圖表分別從機率轉換機制與特徵空間分類兩個視角，說明邏輯斯迴歸的運作原理：

圖 1： 曲線與投影機率（Sigmoid Function & Prediction）

展示模型如何將「線性組合」轉化為「0 到 1 之間的機率值」。

圖 2： 二維空間決策邊界（Decision Boundary）

此圖展示模型在原始特徵空間中，如何劃分兩個類別。

舉例：判斷「銀行審查信用卡貸款：會不會違約」

圖 1：看「評分準不準、核准門檻要設多嚴」

模型會把收入和負債算成一個綜合風險分數，再轉成「違約機率（0%～100%）」。

用途 1：看系統是「很確定」還是「瞎猜」

好的情況：大部分客戶的分數不是 5% 就是 95%，代表系統很肯定誰會過、誰不會過。 差的情況：一堆人都拿到 48%、51%、52%，代表系統根本分不清楚，判斷很猶豫。

用途 2：決定要不要改「通過門檻」

預設是：違約機率超過 50% 就不給過。 景氣大好、銀行想衝業績時，可以看圖放寬到 70% 才拒絕；遇到金融海嘯、寧可錯殺不能賠錢時，看圖把標準收緊到 20% 就直接拒絕。

圖 2：看「拿一把直尺切，管不管用」

橫軸是收入、縱軸是負債比，把每個客戶點上去：正常還款的點藍色，倒債的點紅色。

用途 1：確認資料能不能用「一條直線」分開

如果紅點全聚在左上角（低收入＋高負債），藍點全在右下角（高收入＋低負債），你拿一把直尺畫一條斜線，就能分得很漂亮。這代表用邏輯斯迴歸很合適。 如果紅點全包在藍點的正中間（例如中間收入的人才容易違約），直尺不管怎麼放都會切錯，這張圖告訴你：這不是直線能解決的問題。

用途 2：抓出「例外客戶」

有個藍點明明在高風險區卻沒違約，你可以點開這筆資料，查他是不是有其他隱形資產。